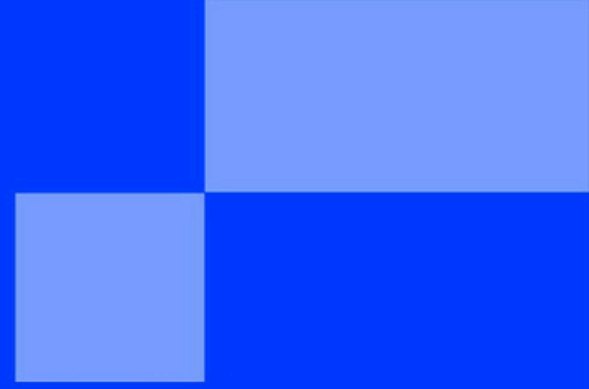


{EPITECH}



AvenDucks



AvenDucks

Contexte:

- Bonjour et bienvenue à Epitech ! Aujourd'hui, tu ne vas pas seulement jouer à un jeu vidéo... tu vas le créer et le réparer !
- Le jeu "**AvenDucks**" est cassé. Notre mascotte est coincée et ne peut pas se défendre contre les Boss.
- Ta mission : enfiler ta casquette de développeur, fouiller dans le code et réparer les mécaniques du jeu.



Le Concept : Code pour débloquer !:

Il y en a pour tous les niveaux (de grand débutant à expert).

Le principe est simple : pour avoir accès à un niveau fonctionnel, tu dois d'abord le coder/compléter ! Plus tu choisis un niveau difficile, plus les blocs de code que tu devras réparer seront complexes.

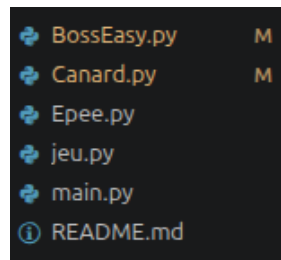
- ● Niveau EASY : Initialisation du Monde
- ● Niveau MEDIUM : Dire brièvement ce que ça va être
- ● Niveau HARD : Dire brièvement ce que ça va être

Prêt(e) ? Suis le guide étape par étape !

PHASE 1 : Préparer ton poste de travail

Avant de coder, un bon développeur doit installer ses outils. C'est très simple :

1. Télécharge les fichiers du jeu : on mettra le dossier du jeu en .zip
2. Dézippe le dossier : Fais un clic droit sur le fichier téléchargé et choisis "Extraire tout". Tu obtiendras un dossier nommé Epiduck_Projet.
3. Ouvre l'éditeur de code : Ouvre le logiciel de code (comme Visual Studio Code présent sur ton ordinateur).
4. Importe le projet : Fais glisser tout ton dossier Epiduck_Projet à l'intérieur du logiciel. Tu devrais maintenant voir la liste de nos fichiers sur la gauche (main.py, Canard.py, jeu.py, etc.).



PHASE 2 : Créer ta "bulle" de développement (Le venv)

- Un vrai développeur Epitech ne code jamais n'importe comment ! Pour éviter d'installer des choses sur tout l'ordinateur, on va créer un **"Environnement Virtuel" (venv)**.

→ Vois ça comme une petite bulle magique et isolée, fabriquée spécialement pour notre jeu Epiduck. Tout ce qu'on va y faire restera à l'intérieur !

- Voici les étapes pour créer et équiper ta bulle :

→ 1. Ouvre le Terminal : C'est le fameux écran noir où les informaticiens tapent des commandes ! Dans ton éditeur de code (en haut de l'écran), trouve le menu Terminal et clique sur Nouveau Terminal.

→ 2. Fabrique la bulle : Dans le terminal qui vient de s'ouvrir en bas, tape cette commande exactement comme ceci, puis appuie sur la touche Entrée :

```
python -m venv env
```

- Un nouveau dossier nommé env vient d'apparaître dans tes fichiers à gauche).

3. Entre dans la bulle (L'activation) : Maintenant que la bulle existe, il faut dire à ton ordinateur de rentrer dedans. Tape la commande qui correspond à ton système :

- Si tu es sur **WINDOWS** : `env\Scripts\activate`
- Si tu es sur **MAC OU LINUX** : `source env/bin/activate`

👉 L'indice de réussite : Regarde bien ton terminal. Si tu vois écrit (env) en vert ou en blanc au tout début de ta ligne de commande, c'est gagné ! Tu es dans la bulle.

4. Installe le moteur du jeu : Il ne reste plus qu'à ranger nos outils dans la bulle.

Pour pouvoir afficher des images, faire bouger des personnages et jouer des sons en Python, on utilise une "bibliothèque" d'outils qui s'appelle Pygame. Pour l'installer, tape cette dernière commande :

```
pip install pygame
```

(Laisse défiler les petites barres de téléchargement...)

Félicitations ! Ton poste de travail est configuré comme celui d'un vrai pro. Tu es prêt(e) à ouvrir les fichiers et à commencer à coder !



```
❖ (venv) jarod@jarod-RX16:~/AvenDucks_AER$
```

● NIVEAU 1 : EASY - Initialisation du Monde

Pour commencer ton aventure, notre canard a besoin d'un monde dans lequel exister. Ton premier objectif est de préparer **la fenêtre de jeu et d'y ajouter un joli décor de fond.**

🔧 Étape 1 : Agrandir la fenêtre du jeu:

- Actuellement, la fenêtre de jeu n'est pas configurée, elle risque d'être trop petite pour afficher notre boss ! Dans Pygame, on utilise une fonction spéciale pour définir la **Largeur (Width)** et la **Hauteur (Height)** de l'écran en pixels.

→ Ta mission : Cherche la ligne qui crée la fenetre. Remplace les chiffres existants pour lui donner **une largeur de 1600 pixels et une hauteur de 600 pixels.**

📖 Lien de documentation Pygame : [pygame.display.set_mode](#)

🖼 Étape 2 : Ajouter le décor (Background):

Un écran noir, c'est un peu triste. Nous avons préparé **une image de fond pour ce niveau**, mais le jeu ne sait pas où la trouver sur l'ordinateur. Tu vas devoir indiquer **le chemin d'accès (le dossier + le nom du fichier)** de l'image.

- Ta mission : Trouve **la variable background**. Entre les guillemets, écris le chemin exact de l'image.
- Indice : Toutes les images sont rangées dans **le dossier assets/**, et l'image de ce niveau s'appelle **backgroundEASY.png**.
- 📖 Lien de documentation Pygame : [pygame.image.load](#)

🦆 Étape 3 : Faire apparaître notre héros

→ Maintenant que nous avons une belle fenêtre et un décor, il est temps de faire entrer notre canard en scène ! Tout comme on a indiqué le chemin pour le décor, on va faire exactement pareil pour notre héros. Mais attention, cette fois-ci ça se passe dans le fichier dédié à notre personnage !

1. Ta mission : Ouvre le fichier **Canard.py**. Cherche les deux lignes qui préparent l'apparence de l'épée et du canard avec "image_base", "image_epee".
2. Entre les guillemets, écris le chemin exact de l'image de notre héros.

🔪 Objectif Final : À l'attaque !

- Ton code est terminé, ton canard est prêt, le décor est planté... Il ne te reste plus qu'à jouer et terminer ce niveau ! Les contrôles sont déjà codés en coulisses.
- Ta mission : Lancer le jeu !

Pour ce faire, retourne dans le terminal où tu as lancé ton 'env' ou 'bulle magique' et tape cette commande:

"python3 main.py"

Déplace-toi vers la droite avec **la touche "D"**, vers la gauche avec **la touche "Q"**, pour trouver le boss.

- Sur ton chemin, n'oublie pas de marcher sur l'épée pour la ramasser. Une fois équipé, approche-toi du boss et appuie sur **la touche "E"** de ton clavier pour l'attaquer. Une fois le boss vaincu, tu peux désormais avancer vers le niveau 2! Place toi dans le dossier 'niveau 2' et c'est parti!

● NIVEAU 2 : MEDIUM - La Physique et la Survie

Bravo ! Ton canard est maintenant dans le monde, mais il est un peu rigide... et surtout, il est **immortel** ! Dans ce niveau, tu vas lui apprendre à subir la gravité, à **tirer des projectiles** et à **afficher sa barre de vie**.

Étape 1 : Dompter la Gravité 🍎:

→ Pour que ton canard puisse **sauter et retomber**, il faut lui appliquer des **lois physiques**. Sans cela, s'il saute, il s'envolera dans l'espace !

→ Ta mission : Ouvre le fichier Canard.py. Dans la fonction **appliquer_gravite**, tu dois compléter deux lignes :

- Faire varier la position verticale (**rect.y**) en fonction de la **vitesse (velocity_y)**.
- Augmenter **la vitesse de chute** grâce à la constante de gravité.
- Indice : En informatique, pour faire descendre un objet, **on ajoute des pixels à sa position Y**.

→ Pour t'aider n'hésite pas à aller voir sur internet de la documentation sur les fonctionnalités pygame!

- Lien de documentation : [Utiliser les Rectangles Pygame](#)

Étape 2 : La trajectoire des projectiles 🚀:

→ Quand ton canard lance une attaque à distance (une balle ou un sort), celle-ci doit avancer toute seule. Mais attention, elle doit savoir si elle va vers la gauche ou vers la droite !

Ta mission : Ouvre le fichier **projectile.py**. Dans la fonction `deplacer`, tu dois **multiplier la vitesse** par la **direction**.

- Le calcul : **vitesse * direction**. Si la direction est 1, elle avance. Si c'est -1, elle recule !

Indice : Utilise la variable `self.direction` pour combler le trou.

Étape 3 : Afficher la barre de vie (Interface) ❤️:

Un combat n'est pas drôle si on ne voit pas les dégâts ! Tu vas coder **l'affichage de la barre de vie** qui rétrécit quand le Boss est touché.

Ta mission : **Ouvre le fichier Boss.py**. Dans la fonction `update_pv`, tu vas dessiner deux rectangles **l'un sur l'autre** :

- Un fond gris pour la vie maximum (la taille totale).
- Une barre verte par-dessus dont la largeur dépend de la vie actuelle.

Indice : Pour le premier trou, utilise `self.max_health`. Pour le deuxième (la barre verte), utilise `self.health`.

Lien de documentation : [Dessiner des rectangles avec `pygame.draw.rect`](#)

🎯 Objectif Final : “Survivre au Duel !” :

Maintenant que ton canard est soumis à la physique et que ton Boss a une barre de vie fonctionnelle, il est temps de passer aux choses sérieuses.

- Relance le jeu : Tape `python3 main.py` dans ton terminal ou avec la petite flèche en haut à droite de ton écran.
- Le test : Essaie de sauter (touche Espace). Est-ce que le canard retombe bien ?
- Le combat : Attaque le Boss Medium. Est-ce que sa barre de vie verte diminue quand tu le touches ?
- Attention : Le Boss Medium est plus rapide que le précédent, utilise tes nouveaux sauts pour esquiver ses attaques ! Une fois vaincu, tu es prêt pour le NIVEAU HARD.

● NIVEAU 3 : HARD - Contrôles Avancés et Armure Lourde

- **Félicitations pour être arrivé jusqu'ici !** Le Boss de ce niveau est redoutable : il porte une **armure lourde** qui le rend insensible aux attaques normales. Pour le vaincre, tu vas devoir configurer de nouvelles touches sur ton clavier et coder une attaque **spéciale (le fameux Bazooka)** capable de percer sa défense grâce aux collisions !

Étape 1 : Prendre le contrôle (Événements Clavier)

→ Pour que le canard obéisse au doigt et à l'œil, **le jeu doit "écouter"** le clavier. En programmation, on appelle cela écouter des **événements**.

Ta mission : Ouvre le fichier qui gère les inputs (**main.py**). Tu dois compléter la détection des touches :

- Indiquer à Pygame qu'on cherche à détecter quand une touche est enfoncée vers le bas.
- Assigner la touche 'E' pour déclencher la fonction du tir normal.
- Assigner la touche 'F' pour déclencher la fonction de l'attaque spéciale.

Indice : **Dans Pygame**, l'action d'appuyer sur une touche s'appelle **KEYDOWN**. Ensuite, chaque touche a un nom de code qui commence par **K_ suivi de la lettre minuscule** (par exemple, la touche espace s'écrit **K_SPACE**).

Lien de documentation : [Gestion des touches avec Pygame](#)

Étape 2 : Casser l'armure (Les Collisions)

Maintenant que tu peux tirer, il faut coder ce qu'il se passe quand les tirs touchent le Boss.

Les tirs normaux doivent se **détruire sans faire de dégâts**, tandis que **les tirs spéciaux doivent exploser** et faire baisser sa barre de vie !

Ta mission : **Dans la fonction verifier_collision**, tu as 5 petits trous à remplir pour rendre la logique parfaite:

- parcourir la **liste des projectiles** (tirs normaux) du canard.
- Si un tir normal touche, utiliser la méthode pour le retirer (**remove**) de la liste.
- Parcourir la liste des **projectiles_speciaux** du canard.
- Vérifier si la **"hitbox"** (le rectangle) du tir spécial entre en collision avec le rectangle du Boss.
- Si c'est le cas, soustraire 100 points à la vie (**health ou pv**) du Boss.

Indice : Pour savoir si deux rectangles se rentrent dedans, Pygame possède une méthode magique qui s'appelle **colliderect()**.

Lien de documentation : [Détection des collisions avec colliderect](#)

🏆 Objectif Final : Le Boss Ultime !

Ton code est complet, la logique de collision est en place et tes touches sont configurées. C'est l'heure du duel final !

- Relance le jeu
- Test de l'armure : Déplace-toi et utilise la touche E. Regarde dans ton terminal, tu devrais voir le message "**L'armure du boss est trop épaisse !**" s'afficher quand tes tirs normaux le touchent.
- L'artillerie lourde : **Appuie sur F** pour dégainer ton attaque spéciale. Vise bien, regarde sa barre de vie chuter et **terrasse ce Boss !**

Mission accomplie ! 🎉

Si tu as réussi à vaincre le Boss Hard, **tu as terminé le projet AvenDucks avec brio.**

Tu as manipulé des variables, de la physique, des interfaces et des collisions.

N'hésite pas à appeler un animateur pour lui montrer ton **chef-d'œuvre**, ou même à **bidouiller** le code pour rendre ton canard encore plus puissant !



v1

{EPITECH}